Simulacro Parcial 1 — PetroSur S.A.

Sep 30, 2026 · @Emilio

Programación IV — UTN FRRe. PetroSur exporta crudo desde Puerto Rosales (Bahía Blanca) con una flota de cuatro buques. Tu programa tiene que decidir qué buque lleva cada pedido, calcular cuándo llega, cuánto cuesta y cuánto se paga de multa. Todo en Node.js, todo por consola.

Reglas

- Duración: 3 horas. Individual.
- Runtime: Node.js 22. Un solo archivo `parcial.js` que se ejecuta con `node parcial.js`.
- Sin API, sin Express, sin paquetes de npm. Toda la salida es por `console.log` / `console.table`.
- Arrancás desde `plantilla.js`: trae los datos ya cargados. **No modifiques el bloque de datos.**
- Se entrega: `parcial.js` + un archivo `respuestas.md` con las Partes 6 y 7.
- Los números del informe se corrigen contra la clave. Un resultado correcto sin código que lo explique no suma; un razonamiento correcto con un error de cuentas suma parcial.
- Habrá una defensa oral corta: tenés que poder explicar cualquier línea de tu código.

Los datos

Todos los buques parten el día 0 desde **Puerto Rosales**. Los datos completos están en `plantilla.js`; estas tablas son para leer el problema.

Puertos (grados decimales; latitud negativa = sur, longitud negativa = oeste)

Puerto	Latitud	Longitud	Factor de ruta
Puerto Rosales (origen)	-38.93	-62.07	1.00
Montevideo	-34.90	-56.21	1.25
Rio Grande	-32.05	-52.10	1.15
Santos	-23.96	-46.33	1.10
Salvador	-12.97	-38.51	1.10

Puerto	Latitud	Longitud	Factor de ruta
Ciudad del Cabo	-33.92	18.42	1.05

Flota

Buque	Capacidad (bbl)	Velocidad (nudos)	Consumo (t/día)	Costo diario (USD)	Calefaccionado
Bermejo	750.000	14,5	38	28.000	no
Paraná	1.000.000	15,0	50	35.000	sí
Pilcomayo	750.000	13,0	42	25.000	no
Iguazú	2.000.000	15,5	75	60.000	sí

Pedidos (plazo = día en que la descarga tiene que estar terminada)

Pedido	Destino	Crudo	Barriles	Plazo (día)
P1	Montevideo	Medanito	900.000	5
P2	Santos	Escalante	1.200.000	9
P3	Rio Grande	Medanito	400.000	4
P4	Ciudad del Cabo	Medanito	1.500.000	16
P5	Salvador	Escalante	800.000	12
P6	Santos	Medanito	700.000	10
P7	Montevideo	Medanito	-50.000	6
P8	Lagos	Medanito	600.000	20

Ojo: no todos los pedidos son válidos.

Reglas del negocio y fórmulas

Distancia. La Tierra es una esfera de radio R = 3440,065 millas náuticas (mn). La distancia en línea recta sobre la esfera (ortodrómica) entre dos puntos se calcula con la fórmula de Haversine, con latitudes φ y longitudes λ **en radianes**:

$$h = \sin^2\left(\frac{\varphi_2 - \varphi_1}{2}\right) + \cos\varphi_1 \cos\varphi_2 \sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right)$$

$$d = 2R \cdot \arcsin\left(\sqrt{h}\right)$$

La distancia de navegación real es d × factorRuta del puerto de destino.

Tiempo. 1 nudo = 1 mn por hora. Días navegando = millas / (velocidad × 24). La vuelta a Puerto Rosales se hace vacío, a la misma velocidad y por la misma ruta.

Un viaje, paso a paso. El buque está libre en Puerto Rosales desde el día `disponibleDesde`.

1. Carga: 1 día.
2. Navega a destino.
3. Descarga: 1 día. Al terminar la descarga se registra `finEntrega`.
4. Vuelve vacío. Al llegar se registra `regreso`, y el buque queda disponible desde ese día.

Crudo. Medanito es liviano: lo lleva cualquier buque. Escalante es pesado: sólo lo llevan buques calefaccionados, que consumen **8 t/día extra** mientras navegan cargados con Escalante (en la vuelta vacía, no).

Costos de un viaje.

- Combustible: toneladas consumidas en la ida + toneladas en la vuelta, a USD 620 la tonelada. Sólo se consume navegando.
- Operativo: (regreso − disponibleDesde) × costoDiario del buque.
- Costo del viaje = combustible + operativo.

Multas. Cada barril vale USD 70. Si un viaje termina de descargar después del plazo del pedido, se paga 1,5 % del valor de los barriles de **ese viaje** por cada día o fracción de atraso. Ejemplo: 0,3 días tarde = 1 día de multa; 2,01 días tarde = 3 días.

Pedidos grandes. Si un pedido supera la capacidad de un buque, se reparte en varios viajes (del mismo buque o de otros). Cada viaje lleva `min(capacidad, barriles que faltan)`.

Parte 1 — Distancias

1. Implementá `distanciaMN(puertoA, puertoB)` con Haversine.
2. Implementá `distanciaRuta(nombreDestino)`: distancia desde Puerto Rosales × factorRuta.
3. Mostrá con `console.table` una fila por destino: ortodrómica y de ruta, redondeadas a 2 decimales.

Control: Puerto Rosales → Montevideo da 370,90 mn ortodrómicas y 463,62 mn de ruta. Si no te da eso, no sigas.

Parte 2 — Modelo

1. Una clase `Buque` con:
   - la agenda de viajes y el día `disponibleDesde` como **campos privados** (`#`), expuestos sólo por getters;
   - un método `puedeTransportar(crudo)`;
   - un método `consumoDiario(cargado, crudo)`;
   - un método `cotizar(pedido, barriles)` que calcula el viaje **sin registrarlo** (salida, finEntrega, regreso, toneladas, costo, multa);
   - un método `registrar(cotizacion)` que lo agrega a la agenda y actualiza `disponibleDesde`.
2. Una clase `BuqueCalefaccionado` que **extiende** `Buque` y sobreescribe `puedeTransportar` y `consumoDiario`. El consumo extra se calcula a partir del consumo de la clase base (`super`), no copiando la fórmula.
3. El constructor de `Buque` rechaza capacidad o velocidad no positivas.
4. Una clase de error propia `PedidoInvalidoError`. Un pedido es inválido si sus barriles o su plazo no son números positivos, o si su destino no existe.

Parte 3 — Planificación

Implementá `planificar(flota, pedidos)` siguiendo **exactamente** este algoritmo:

1. Validá cada pedido. Los inválidos van a una lista de rechazados con su motivo; el programa **no se corta**. Un error que no sea `PedidoInvalidoError` sí debe propagarse.
2. Ordená los válidos por plazo ascendente; a igual plazo, primero el de más barriles.
3. Para cada pedido, mientras falten barriles:
   a. Candidatos = buques que pueden llevar ese crudo.
   b. Cotizá el viaje en cada candidato con `min(capacidad, faltantes)` barriles.
   c. Elegí el que termina de descargar antes (`finEntrega` menor). Si empatan, el de menor costo.
   d. Registrá el viaje en ese buque y descontá los barriles.

Parte 4 — Informe por consola

Imprimí, en este orden:

1. Pedidos rechazados, con su motivo.
2. Tabla de viajes: buque, pedido, destino, barriles, salida, finEntrega, regreso, toneladas, costo, multa.

3. Tabla de pedidos: plazo, día de entrega completa (el último viaje), cantidad de viajes, EN PLAZO / ATRASADO, multa total.
4. Tabla de flota: viajes, días ocupado y utilización = días ocupado / horizonte, donde horizonte es el último regreso de toda la operación.
5. Totales: barriles entregados, costo logístico, multas, costo total y costo por barril.

Parte 5 — Simulación asíncrona

Reproducí la operación en "tiempo real": **1 día simulado = 40 ms**.

1. Escribí `esperar(ms)` que devuelve una Promise resuelta con `setTimeout`.
2. Escribí `async function simularBuque(buque, inicio)` que recorre los viajes del buque en orden y, para cada uno, imprime tres eventos en su momento: zarpa (salida + 1), termina de descargar (finEntrega) y vuelve a Puerto Rosales (regreso). Formato: `[día 5.80] Bermejo termina de descargar en Santos`.
3. Los cuatro buques se simulan **en paralelo** con `Promise.all`. Al terminar, imprimí qué buques finalizaron.
4. La bitácora completa tiene que salir en orden cronológico aunque los eventos vengan de buques distintos. Tu programa debe verificarlo e imprimir si se cumplió.
5. La llamada que arranca la simulación va **antes** de un `console.log("planificación terminada")` al final del archivo. No uses `await` en el nivel superior.

Parte 6 — Predicción y diagnóstico

Sin ejecutar: escribí la salida exacta de cada fragmento y **por qué**. Recién después ejecutalo y, si fallaste, explicá qué parte de tu modelo mental estaba mal. Se corrige la explicación, no sólo el acierto.

6.1

```
console.log(calcularFlete(100));
console.log(tarifa);
var tarifa = 5;
function calcularFlete(millas) { return millas * tarifaBase; }
let tarifaBase = 2;
```

6.2

```
console.log("A: empieza la carga");
setTimeout(() => console.log("B: zarpa"), 0);
Promise.resolve().then(() => console.log("C: papeles aprobados"));
process.nextTick(() => console.log("D: aduana"));
console.log("E: termina la carga");
```

6.3 ¿Qué número aproximado imprime? ¿Qué línea sale primero?

```
const inicio = Date.now();
setTimeout(() => console.log(`zarpó a los ${Date.now() - inicio} ms`), 100);
while (Date.now() - inicio < 500) { /* recalculando rutas... */ }
console.log("rutas recalculadas");
```

6.4

```
class Buque { consumoDiario() { return 40; } }
class BuqueCalefaccionado extends Buque {
  consumoDiario() { return super.consumoDiario() + 8; }
}
const b = new BuqueCalefaccionado();
Buque.prototype.consumoDiario = function () { return 100; };
console.log(b.consumoDiario());
console.log(Object.getPrototypeOf(Object.getPrototypeOf(b)) ===
Buque.prototype);
```

6.5

```
class Buque {
  #combustible = 500;
  get combustibleReal() { return this.#combustible; }
}
const b = new Buque();
b.combustible = 9999;
b["#combustible"] = 1;
console.log(b.combustible, b.combustibleReal);
console.log(Object.keys(b));
```

6.6 Diagnóstico. Un cliente manda su configuración junto con el pedido y el sistema la mezcla así. Qué imprime, qué vulnerabilidad tiene, qué consecuencia tendría en PetroSur (pensá en el Escalante) y cómo lo corregirías.

```
function mezclar(destino, origen) {
  for (const k in origen) {
    if (typeof origen[k] === "object") {
      destino[k] ??= {};
      mezclar(destino[k], origen[k]);
    } else {
      destino[k] = origen[k];
    }
  }
  return destino;
}
const configCliente = JSON.parse('{"prioridad": "alta", "__proto__":
{"calefaccionado": true}}');
const config = mezclar({}, configCliente);

const bermejo = { nombre: "Bermejo", capacidad: 750000 };
console.log(config.prioridad);
console.log(bermejo.calefaccionado);
```

Parte 7 — Justificación escrita

Respondé en `respuestas.md`. Máximo 10 renglones por pregunta. Toda afirmación se apoya en un número que sale de tu programa.

7.1 ¿El algoritmo es bueno? Mirá la tabla de flota y la de pedidos. ¿El algoritmo goloso de la Parte 3 da la mejor asignación posible? Si no, mostrá con tu propia salida una decisión concreta que después "le salió cara" al sistema.

7.2 La propuesta del compañero. Un compañero dice: "los calefaccionados son los únicos que llevan Escalante; entonces el Medanito tiene que ir en buques comunes siempre que alguno llegue en plazo". No opines: implementalo como variante de `planificar`, corré las dos versiones y compará multas y costo total. ¿Tenía razón? ¿Por qué pasa lo que pasa?

7.3 El orden de la bitácora. ¿Qué garantiza que tu bitácora salga en orden cronológico? ¿Lo garantiza `setTimeout`? ¿Qué pasaría si cada espera se calculara relativa al evento anterior del mismo buque en vez de relativa al `inicio`? ¿Y si en medio de la simulación corriera un cálculo sincrónico de 300 ms?

7.4 Bonus: navegar más lento. El consumo diario crece con el cubo de la velocidad:

```
consumo(v) = consumo_{nominal} \cdot \left(\frac{v}{v_{nominal}}\right)^3
```

1. Demostrá que el combustible total de un tramo de d millas es proporcional a v^2.

2. El Bermejo lleva P6 a Santos con días de sobra. Si hace la ida a 10 nudos (la vuelta sigue igual): ¿cuántas toneladas y USD ahorra? ¿Sigue en plazo?
3. ¿Conviene? Considerá todo lo que cambia, no sólo el combustible.

Puntaje

Se aprueba con 60 de 100. El bonus suma hasta 10 puntos extra.

Parte	Qué se evalúa	Puntos
1 — Distancias	Haversine correcto, conversión a radianes, factor de ruta	10
2 — Modelo	Campos privados, herencia con `super`, validación, error propio	20
3 — Planificación	Algoritmo exacto, reparto de pedidos grandes, rechazos sin cortar el programa	20
4 — Informe	Tablas completas y totales correctos	10
5 — Simulación	Promesas, `Promise.all`, reloj absoluto, orden verificado	15
6 — Predicción y diagnóstico	2,5 por fragmento; la explicación vale más que el acierto	15
7 — Justificación	7.1: 3 · 7.2: 4 · 7.3: 3	10
7.4 — Bonus	Demostración + cálculo + decisión	+10